

Technical Report

Design and Verification of a Structural 4-bit Ripple Carry Adder

Reece Dobro

May 25, 2026

Abstract

This report discusses the process of developing and testing a 4-bit adder in the Verilog programming language using AMD Vivado. The 4-bit adder's structure is made up of a 4-bit ripple carry adder modeled using four 1-bit full adder modules written at the gate-level. Testing and validation consist of four non-trivial 4-bit values with reasoning for the choices included in this report. Successful addition of all 4 test values demonstrates the 4-bit adder's reliability in both its programming and structure.

1 Introduction, Background, and Theory

The goal of this project is to program and test a 4-bit ripple carry adder in Verilog through structural-level and gate-level descriptions. The program, *full_adder_struct.v*, is made up of two modules and a testbench that was run through AMD Vivado. The first module is a single bit full adder, while the second module serves as a 4-bit ripple carry adder. The testbench allows for the adder to be tested and deemed correct using 4 chosen test values. This project demonstrates the implementation of basic arithmetic in Verilog and the methodology for verifying designs using a testbench and certain input cases.

The full adder module, *full_adder_struct*, was implemented at the gate-level using 2 XOR gates, 2 AND gates, and a single OR gate. The resulting gate implementation in Verilog of this design is shown in Figure 1. The full adder module has three inputs, (*a*, *b*, and *carry-in*) and two outputs (*sum* and *carry-out*). All of these ports are 1-bit wide. Due to addition being commutative, the order of the inputs (*a*, *b*, and *carry-in*) does not affect the outputs of the module. For example, *a* = 1, *b* = 0, *carry-in* = 0 and *a* = 0, *b* = 1, *carry-in* = 0 would produce the same output, with *sum* = 1 and *carry-out* = 0.

```
xor u1(net1,A,B);
xor u2(S,net1,Cin);
and u3(net2,A,B);
and u4(net3,Cin,net1);
or u5(Cout,net3,net2);
```

Figure 1: Gate-level implementation of the Full Adder in Verilog

Four instances of full adders are used to implement a single 4-bit ripple-carry adder in the module, *rca_4bit*. The ripple carry adder module has two inputs, *a* and *b* which are both 4-bit unsigned vectors. It also has two outputs: *sum*, an unsigned vector, and *carry-out*, a 1-bit signal that indicates whether inputs *a* and *b* generate a carry. The 4-bit ripple carry adder works by using the carry-out from the first full adder (which adds the LSBs of *a* and *b* to produce the LSB of the sum) as the carry-in for the second full adder. The carry-out of the second adder is then used as the carry-in for the third adder. This process continues until the fourth full adder is reached, whose carry-out value represents the overall carry-out of the two 4-bit numbers being added. The fourth adder's sum output is the MSB of the 4-bit sum. The carry-in value of the first full adder instance is 0 because there is no previous instance from which to receive a carry-out value. In order to confirm the success of the ripple carry adder, four sets of test values for *a* and *b* were given. These values and their expected outputs are shown in Table 1. If the ripple carry adder's outputs matched the correct outputs, then the ripple carry adder will have shown to be successful.

Inputs		Expected Output	
a (4-bit)	b (4-bit)	Sum (4-bit)	C_{out}
0000	0000	0000	0
0001	1111	0000	1
0000	0001	0001	0
1111	1111	1110	1

Table 1: 4-Bit Ripple-Carry Adder Test Values

1.1 Justification of Test Values

The rationale for each test case varies, with the primary focus on individual full-adder behavior, carry propagation, and general edge cases. The first test value, $a=0000$ and $b=0000$, was chosen because, if incorrect, it could be used to identify which of the four full adder instances was configured incorrectly. The placement of a single or multiple 1-bits instead of 0-bits in the output would likely reveal this location. This test is not focused on carry values, unlike tests 2 and 4. The second test value, $a=0001$ and $b=1111$, was chosen to verify that the carry-out from each of the four full adder instances is correct. If this test fails while tests 1 and 3 pass, it would likely indicate that the carry-out logic in the design is configured incorrectly. The third test value, $a=0000$ and $b=0001$, was chosen because, if it were incorrect while tests 1 and 4 pass, it could indicate that the 4-bit vector for a or b as an input was improperly configured. If the output is 0000, it is likely that the 4-bit vector a is being used for both input vectors. If the output is 0010, it is likely that the 4-bit vector b is being used for both input vectors. The fourth test value, $a=1111$ and $b=1111$, was chosen to verify that the 4-bit ripple-carry adder can correctly handle the maximum input values and produce the correct sum and carry-out.

2 Results and Analysis

The output of the ripple carry adder, (the 4-bit vector *sum* and the 1-bit port *carry-out*) are shown through Figures 2 and 3. Figure 2 shows the waveform output from AMD Vivado of the testbench and Figure 3 shows the output of the testbench through the TCL Console. These figures provide two different views of the outputs. Figure 2, the output of the waveforms, demonstrates how the changes in the test values cause a change in the *sum* and *carry-out* values. Figure 3 allows for the raw outputs and test values to be analyzed and compared to the expected outputs of the program. The data generated from Figure 3 was compiled into Table 2, along with the values from Table 1, for simplicity and comparison.

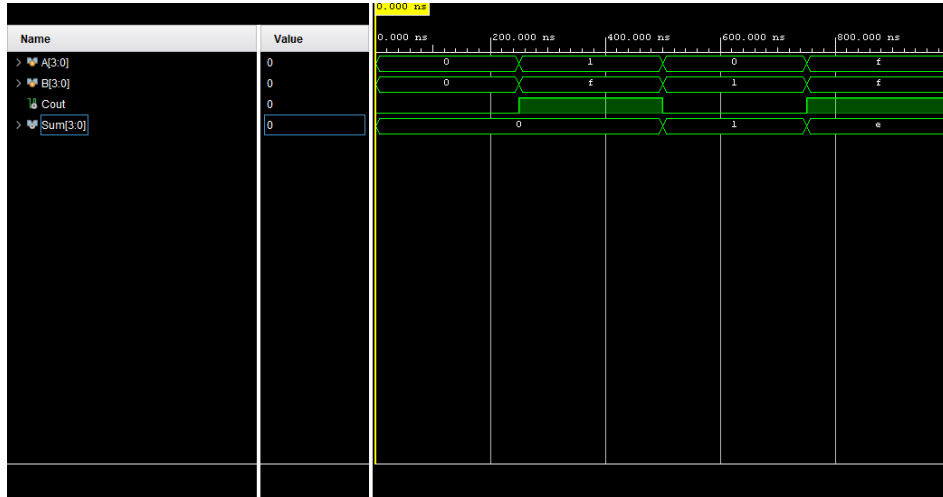


Figure 2: Waveforms Output of the Testbench

```

A: 0000 B: 0000 Sum: 0000 Cout: 0
A: 0001 B: 1111 Sum: 0000 Cout: 1
A: 0000 B: 0001 Sum: 0001 Cout: 0
A: 1111 B: 1111 Sum: 1110 Cout: 1

```

Figure 3: TCL Console Output of the Testbench

Inputs		Program Output		Expected Output	
a (4-bit vector)	b (4-bit vector)	Sum	C_{out}	Sum	C_{out}
0000	0000	0000	0	0000	0
0001	1111	0000	1	0000	1
0000	0001	0001	0	0001	0
1111	1111	1110	1	1110	1

Table 2: 4-Bit Ripple-Carry Adder Verification

As shown by Table 2, the output of the ripple-carry adder matches the expected output. This successful verification demonstrates that the adder is properly implemented and functions as intended. By passing the given test values, the adder is shown to correctly handle edge cases, carry propagation, and maximum input conditions.

A potential weak point in the data is that only 4 test cases are used. Since there are 2^4 possible combinations for each of the 4-bit vectors a and b , there are 2^8 possible test values, with the four sets chosen covering only around 1.5% of the total possible test values. However, it would be impractical to use all possible test values to test out the completed module due to the length of time to manually verify and document each output.

As the outputs from the module matched the expected results, the Verilog program and its logical structure (down to the gate-level) are deemed successful.

3 Conclusions

This project demonstrated how logical functions, as well as gate-level and structural designs, can be implemented and tested in Verilog. It also presented a design for a 4-bit ripple-carry adder constructed from multiple full adders. However, while the program was successful, there are areas that could be improved for future design iterations. The usage of a full adder for the first instance of the ripple carry adder could be replaced by using a half adder, which would reduce the number of gates and transistors for a physical design. Additionally, the test cases focused solely on functional correctness. Time and efficiency were not taken into account, and separate testing should be conducted to determine what changes could be made to the design to optimize performance. A possible future study could take into account propagation path delays and routing efficiency when designing test cases. With that being said, this project was able to demonstrate the capabilities of Verilog to test basic gate-level and structural-level logic, while also creating a module that will be useful for future architecture integrations. Instead of needing to develop at the physical prototyping level, which would require a numerous amount of resources such as discrete logic gates, wires, and chips, a design can be fully tested and implemented digitally with minimal use of physical resources.

This report was compiled using L^AT_EX .